

DAY 21 — MONITORING (LATENCY + TRACE LOGGING)

GOAL

Upgrade from: Black box system 

to: Observable system  (know what happened at each step)

STEP 1 — CREATE MONITOR FILE

 **NEW FILE:** utils/monitor.py

ADD CODE

```
import time

class Monitor:

    def init(self):

        self.logs = []

    def start(self, name):

        return {"name": name, "start": time.time()}

    def end(self, entry):

        entry["end"] = time.time()

        entry["duration"] = entry["end"] - entry["start"]

        self.logs.append(entry)

    def report(self):

        print("\n---- MONITOR ----")

        for log in self.logs:

            print(f"{log['name']} → {log['duration']:.4f}s")

        print("-----\n")
```

STEP 2 — INTEGRATE INTO AGENT : app/agent.py

ADD IMPORT

 **FIND:** from models.llm import generate

✓ **ADD BELOW:** from utils.monitor import Monitor

✓ **MODIFY __init__**

🔧 **FIND:** self.memory = ChatMemory()

✓ **ADD AFTER:** self.monitor = Monitor()

✓ ✓ **STEP 3 — ADD MONITORING IN run()**

✓ **FIND THIS (start of run):** async def run(self, query):

✓ **ADD BELOW IT:** self.monitor.logs = [] # reset per request

✓ ✓ **STEP 4 — MONITOR PLANNER**

🔧 **FIND:** plan = self.planner.create_plan(enriched_query)

✓ **REPLACE WITH**

```
t = self.monitor.start("Planner")  
  
plan = self.planner.create_plan(enriched_query)  
  
self.monitor.end(t)
```

✓ ✓ **STEP 5 — MONITOR TOOL EXECUTION**

🔧 **FIND THIS LOOP**

```
for tool, task in plan:  
  
    tasks.append(self.execute_tool(tool, task))
```

✓ **Replace EACH tool call inside execute_tool() OR wrap at result stage:**

🔧 **FIND:** results_raw = await asyncio.gather(*tasks)

✓ **REPLACE WITH**

```
t = self.monitor.start("Tool Execution")
```

```
results_raw = await asyncio.gather(*tasks)
```

```
self.monitor.end(t)
```

✅✅ STEP 6 — MONITOR FINAL LLM

🔧 **FIND:** `final_answer = generate(final_prompt)`

✅ REPLACE WITH

```
t = self.monitor.start("Final LLM")
```

```
final_answer = generate(final_prompt)
```

```
self.monitor.end(t)
```

✅✅ STEP 7 — PRINT REPORT

🔧 **FIND (before return):** `return final_answer`

✅ **ADD BEFORE IT:** `self.monitor.report()`

✅✅ FINAL FLOW NOW

User Query

↓

Planner (timed ✅)

↓

Tools (timed ✅)

↓

LLM (timed ✅)

↓

Monitor report ✅

✅✅ TEST

Run: `uvicorn app.api:app --reload`

✓ **Example output:**

---- MONITOR ----

Planner → 0.32s

Tool Execution → 1.85s

Final LLM → 2.40s

✓ ✓ **WHAT YOU BUILT**

Observability layer

✓ ✓ **SYSTEM LEVEL NOW**

✓ AI system

✓ API + Docker

✓ Async execution

✓ Caching

✓ Memory (vector)

✓ Streaming

✓ ✓ Monitoring (NEW)

✓ ✓ **WHY THIS IS BIG**

Now you can:

- detect slow components
 - debug issues
 - optimize performance
 - trace full pipeline
-

✓ ✓ **INDUSTRY TERM**

Tracing / Observability

Used in:

- LangSmith
 - OpenTelemetry
 - Production AI systems
-

✅✅ **NEXT**

Day 22 → Advanced Evaluation (automatic scoring + benchmark)

If ready: 👉 say **Day 22**